COMP2025 Introduction to Data Science — Assignment 1

Pham Minh Khoi

2026-06-20

Declaration

By submitting this work I certify that: no part of it has been copied from another student or any other source except where acknowledged; no part was written or produced for me by another person except as authorised by the lecturer; I have disclosed all use of generative-AI tools in the Reflection Log; and I understand this work may be checked by plagiarism- and AI-detection software.

Signed (typed name): Pham Minh Khoi
Student ID: 22145599
Date: 2026-07-15

0. Setup and Reproducibility

This section records the R environment, required packages, personalised seed, and data files used in this report. These details make the analysis reproducible, especially for random procedures such as cross-validation and train-test splitting.

a) R environment and packages

```
library(boot)
library(pROC)
```

```
R.version.string
```

```
## [1] "R version 4.6.0 (2026-04-24 ucrt)"
```

```
packageVersion("boot")
```

```
## [1] '1.3.32'
```

```
packageVersion("pROC")
```

```
## [1] '1.19.0.1'
```

The output above records the R version and package versions used in this analysis. The `boot` package is used later for 10-fold cross-validation, while the `pROC` package is used later to produce the ROC curve and calculate AUC for the classification model.

b) Personalised seed

```r
D <- 599
set.seed(D)

D
```

```
## [1] 599
```

The personalised seed value is **599**, which is the last three digits of Student ID **22145599**. The command `set.seed(599)` is used wherever a random procedure is required. In this assignment, the seed affects the 10-fold cross-validation in Question 1, the 80/20 train-test split in Question 2, and the 10-fold cross-validation in Question 2.

c) Data files used

```r
energy_file <- "D:/data-portfolio/07_learner_retention_analytics_r/data/energy_homes.csv"
learners_file <- "D:/data-portfolio/07_learner_retention_analytics_r/data/learners.csv"

file.exists(energy_file)
```

```
## [1] TRUE
```

```r
file.exists(learners_file)
```

```
## [1] TRUE
```

Both required data files were found successfully, as shown by the two `TRUE` outputs. The file `energy_homes.csv` is used for Question 1, which is the regression task predicting `heating_load`. The file `learners.csv` is used for Question 2, which is the classification task predicting whether a learner completed the course.

1. Question 1 — Regression

This question uses `energy_homes.csv` to model `heating_load`, which is the annual heating energy demand per unit floor area. The goal is to understand how building design variables are related to heating demand.

1.1 Exploration and Data Thinking

a) Data import, dimensions, and missing values

```r
energy <- read.csv("D:/data-portfolio/07_learner_retention_analytics_r/data/energy_homes.csv")

dim(energy)
```

```
## [1] 768  12
```

```r
names(energy)
```

```
##  [1] "home_id"              "relative_compactness" "surface_area"
##  [4] "wall_area"            "roof_area"            "overall_height"
##  [7] "orientation"          "glazing_area"         "glazing_distribution"
## [10] "wall_insul_rating"    "window_age"           "heating_load"
```

```
str(energy)
```

```
## 'data.frame':    768 obs. of  12 variables:
##  $ home_id             : chr  "H1000" "H1001" "H1002" "H1003" ...
##  $ relative_compactness: num  0.684 0.85 0.788 0.753 0.748 0.905 0.946 0.684 0.855 0.727 ...
##  $ surface_area        : num  529 686 788 734 693 ...
##  $ wall_area           : num  281 330 352 249 298 ...
##  $ roof_area           : num  153 132 181 133 141 ...
##  $ overall_height      : num  7 3.5 3.5 3.5 7 7 3.5 3.5 3.5 3.5 ...
##  $ orientation         : int  3 2 5 2 5 4 4 2 2 2 ...
##  $ glazing_area        : num  0 0.1 0.25 0.1 0.25 0.4 0.1 0.1 0.25 0 ...
##  $ glazing_distribution: int  0 5 1 5 4 4 5 2 3 1 ...
##  $ wall_insul_rating   : num  3.87 2.69 2.97 1.91 2.72 4.96 3.6 4.54 4.46 3.86 ...
##  $ window_age          : int  3 16 20 15 15 11 9 5 19 4 ...
##  $ heating_load        : num  85.4 76.9 86.4 85.1 100.7 ...
```

```
colSums(is.na(energy))
```

```
##              home_id relative_compactness         surface_area
##                    0                    0                    0
##            wall_area           roof_area       overall_height
##                    0                    0                    0
##          orientation         glazing_area glazing_distribution
##                    0                    0                    0
##    wall_insul_rating           window_age         heating_load
##                    0                    0                    0
```

The `energy_homes.csv` dataset contains **768 rows** and **12 columns**. Each row represents one simulated residential building. The columns include one identifier variable, ten predictor variables, and one response variable, `heating_load`. The missing-value check shows that every column has **0 missing values**, so no rows need to be removed and no imputation is required.

```
response <- "heating_load"

predictors <- c(
  "relative_compactness",
  "surface_area",
  "wall_area",
  "roof_area",
  "overall_height",
  "orientation",
  "glazing_area",
  "glazing_distribution",
  "wall_insul_rating",
  "window_age"
)

response
```

```
## [1] "heating_load"
```

```
predictors
```

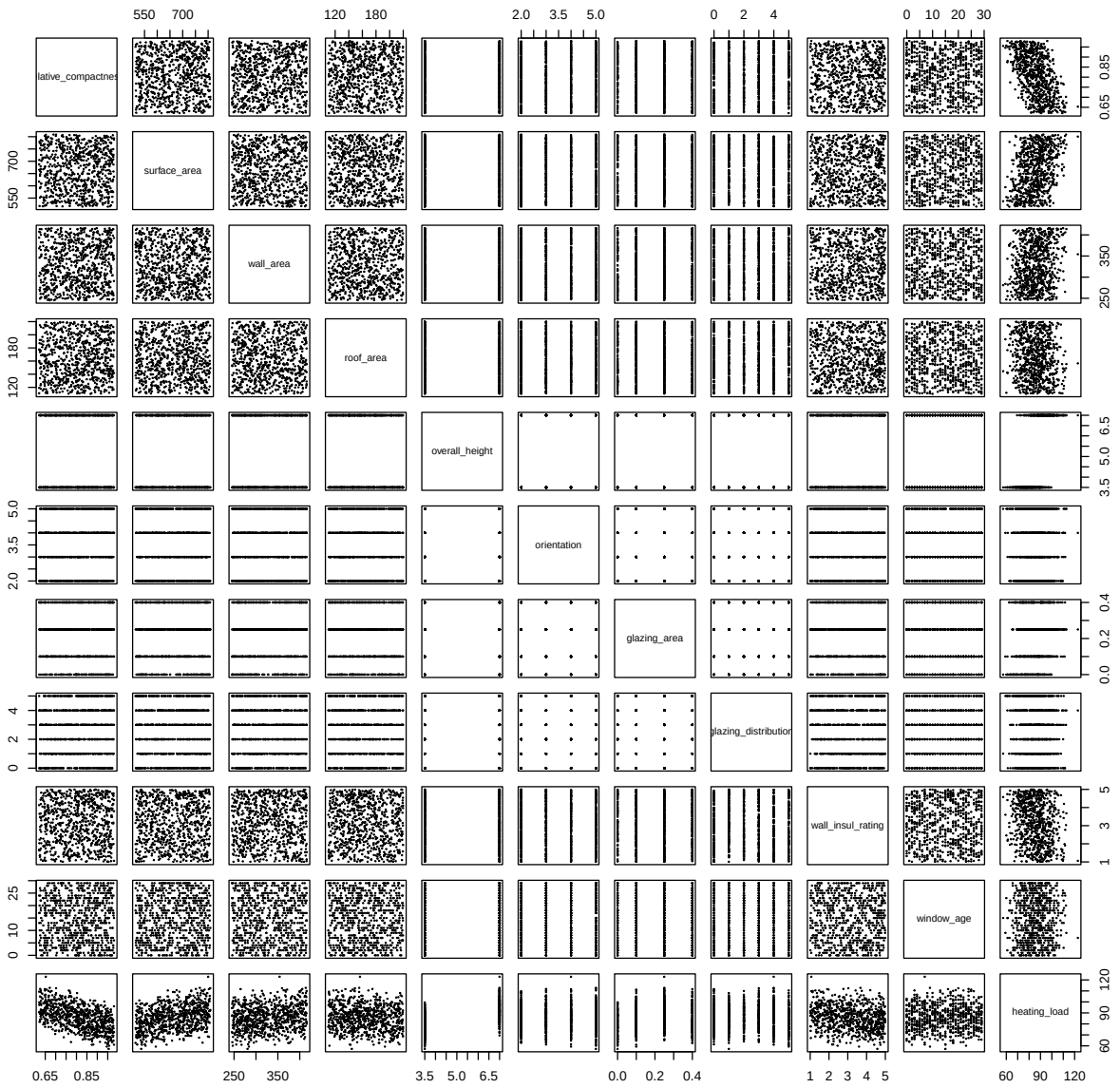
```
##  [1] "relative_compactness" "surface_area"         "wall_area"
##  [4] "roof_area"            "overall_height"       "orientation"
##  [7] "glazing_area"         "glazing_distribution" "wall_insul_rating"
## [10] "window_age"
```

The response variable is `heating_load`, which measures annual heating demand in kWh/m². The variable `home_id` is only a building identifier, so it is not used as a predictor. The ten predictors used in this question are the building design variables listed in the output above.

b) Scatterplot matrix, correlation matrix, and key relationships

```
energy_for_plots <- energy[, c(predictors, response)]

pairs(
  energy_for_plots,
  main = "Scatterplot Matrix of Heating Load and Predictors",
  pch = 19,
  cex = 0.18,
  cex.labels = 0.7
)
```



The scatterplot matrix provides an initial visual overview of the relationships between `heating_load` and the ten predictors. Because several predictors are coded or discrete variables, such as `orientation`, `overall_height`, `glazing_area`, and `glazing_distribution`, some panels appear as vertical or horizontal bands rather than smooth clouds of points. This is expected and does not indicate an error.

```
cor_matrix <- cor(energy_for_plots)

round(cor_matrix, 3)
```

```
##                      relative_compactness surface_area wall_area roof_area
## relative_compactness                1.000        0.023     0.055     0.049
## surface_area                        0.023        1.000    -0.005    -0.010
## wall_area                           0.055       -0.005     1.000    -0.008
## roof_area                           0.049       -0.010    -0.008     1.000
```

```
## overall_height                    -0.007       -0.012    -0.007    -0.060
## orientation                        0.053       -0.003    -0.023    -0.005
## glazing_area                      -0.014        0.027     0.056    -0.097
## glazing_distribution               0.079       -0.057     0.043    -0.004
## wall_insul_rating                  0.014        0.009     0.004    -0.005
## window_age                         0.011        0.036    -0.008    -0.011
## heating_load                      -0.520        0.328     0.172    -0.104
##                       overall_height orientation glazing_area
## relative_compactness          -0.007       0.053       -0.014
## surface_area                  -0.012      -0.003        0.027
## wall_area                     -0.007      -0.023        0.056
## roof_area                     -0.060      -0.005       -0.097
## overall_height                 1.000      -0.002        0.002
## orientation                   -0.002       1.000        0.073
## glazing_area                   0.002       0.073        1.000
## glazing_distribution          -0.014      -0.038       -0.016
## wall_insul_rating              0.011      -0.062       -0.012
## window_age                     0.039       0.014        0.022
## heating_load                   0.647       0.009        0.148
##                       glazing_distribution wall_insul_rating window_age
## relative_compactness                 0.079             0.014      0.011
## surface_area                        -0.057             0.009      0.036
## wall_area                            0.043             0.004     -0.008
## roof_area                           -0.004            -0.005     -0.011
## overall_height                      -0.014             0.011      0.039
## orientation                         -0.038            -0.062      0.014
## glazing_area                        -0.016            -0.012      0.022
## glazing_distribution                 1.000             0.012      0.009
## wall_insul_rating                    0.012             1.000     -0.007
## window_age                           0.009            -0.007      1.000
## heating_load                        -0.053            -0.222      0.033
##                       heating_load
## relative_compactness        -0.520
## surface_area                 0.328
## wall_area                    0.172
## roof_area                   -0.104
## overall_height               0.647
## orientation                  0.009
## glazing_area                 0.148
## glazing_distribution        -0.053
## wall_insul_rating           -0.222
## window_age                   0.033
## heating_load                 1.000
```

```r
cor_with_heating <- cor_matrix["heating_load", predictors]

round(cor_with_heating, 3)
```

```
## relative_compactness         surface_area            wall_area
##               -0.520                0.328                0.172
##            roof_area       overall_height          orientation
##               -0.104                0.647                0.009
##         glazing_area glazing_distribution    wall_insul_rating
##                0.148               -0.053               -0.222
```

```
##           window_age
##                0.033
```

```r
ranked_correlations <- sort(abs(cor_with_heating), decreasing = TRUE)

round(ranked_correlations, 3)
```

```
##       overall_height relative_compactness         surface_area
##                0.647                0.520                0.328
##    wall_insul_rating            wall_area         glazing_area
##                0.222                0.172                0.148
##            roof_area glazing_distribution           window_age
##                0.104                0.053                0.033
##          orientation
##                0.009
```

Based on the correlations with `heating_load`, the strongest linear relationship is with `overall_height`, with a correlation of approximately **0.647**. This positive correlation suggests that taller buildings tend to have higher heating load in this dataset. The second strongest relationship is with `relative_compactness`, with a correlation of approximately **-0.520**, meaning more compact buildings tend to have lower heating load. By contrast, `orientation` has a correlation of approximately **0.009**, which is almost zero, so it appears to have essentially no linear relationship with heating demand. `window_age` is also very weak, with a correlation of approximately **0.033**.

c) Bias–variance discussion

A model using only the strongest predictor, `overall_height`, would likely have **relatively high bias but low variance**. It would have low variance because a one-predictor model is simple and its predictions are less sensitive to small changes in the training data. However, it would likely have high bias because `heating_load` is also related to other predictors, especially `relative_compactness`, `surface_area`, and `wall_insul_rating`, so using only `overall_height` would ignore important building-design information.

1.2 Simple Linear and Polynomial Regression

a) Simple linear regression model and estimated equation

```r
simple_model <- lm(heating_load ~ relative_compactness, data = energy)

summary(simple_model)
```

```
##
## Call:
## lm(formula = heating_load ~ relative_compactness, data = energy)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -24.0994  -7.1165  -0.5538   6.9866  28.5460
##
## Coefficients:
##                      Estimate Std. Error t value Pr(>|t|)
## (Intercept)           130.871      2.702   48.44   <2e-16 ***
## relative_compactness  -55.906      3.318  -16.85   <2e-16 ***
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 9.428 on 766 degrees of freedom
## Multiple R-squared:  0.2704, Adjusted R-squared:  0.2695
## F-statistic:   284 on 1 and 766 DF,  p-value: < 2.2e-16
```

```
coef(simple_model)
```

```
##          (Intercept) relative_compactness
##            130.87071            -55.90607
```

A simple linear regression model was fitted to predict `heating_load` from `relative_compactness`. The estimated regression equation is:

Predicted heating_load = 130.871 - 55.906 * relative_compactness

This means that the predicted heating load decreases as `relative_compactness` increases. This model is used as a baseline because it uses only one predictor to explain heating demand.

b) Hypothesis test and slope interpretation

For the slope of `relative_compactness`, the hypotheses are:

H0: beta_1 = 0. There is no linear relationship between `relative_compactness` and `heating_load`.

H1: beta_1 is not equal to 0. There is a non-zero linear relationship between `relative_compactness` and `heating_load`.

```
summary(simple_model)$coefficients
```

```
##                       Estimate Std. Error   t value      Pr(>|t|)
## (Intercept)          130.87071   2.701524  48.44329 2.023468e-235
## relative_compactness -55.90607   3.317641 -16.85115  1.960585e-54
```

The p-value for `relative_compactness` is approximately **1.96e-54**, which is far below 0.05. Therefore, the null hypothesis is rejected. There is strong statistical evidence that `relative_compactness` is a significant linear predictor of `heating_load`.

The slope for `relative_compactness` is approximately **-55.906**. This means that a one-unit increase in `relative_compactness` is associated with an estimated **55.906 kWh/m² decrease** in annual heating demand. Since a full one-unit increase is large on the compactness scale, a more practical interpretation is that a 0.1 increase in `relative_compactness` is associated with an estimated decrease of about **5.591 kWh/m²**.

c) Standard error and 95% confidence interval

```
summary(simple_model)$coefficients["relative_compactness", "Std. Error"]
```

```
## [1] 3.317641
```

```
confint(simple_model)["relative_compactness", ]
```

```
##     2.5 %    97.5 %
## -62.41882 -49.39332
```

The standard error of the slope is approximately **3.318**. The 95% confidence interval for the slope is approximately [**-62.419, -49.393**].

This confidence interval does not include zero, which supports the conclusion that `relative_compactness` is statistically significant. The interval is reasonably narrow relative to the size of the slope, suggesting that the estimate is fairly precise.

d) R^2, residual standard error, and adequacy for design decisions

```
summary(simple_model)$r.squared
```

```
## [1] 0.2704493
```

```
summary(simple_model)$adj.r.squared
```

```
## [1] 0.2694968
```

```
summary(simple_model)$sigma
```

```
## [1] 9.427667
```

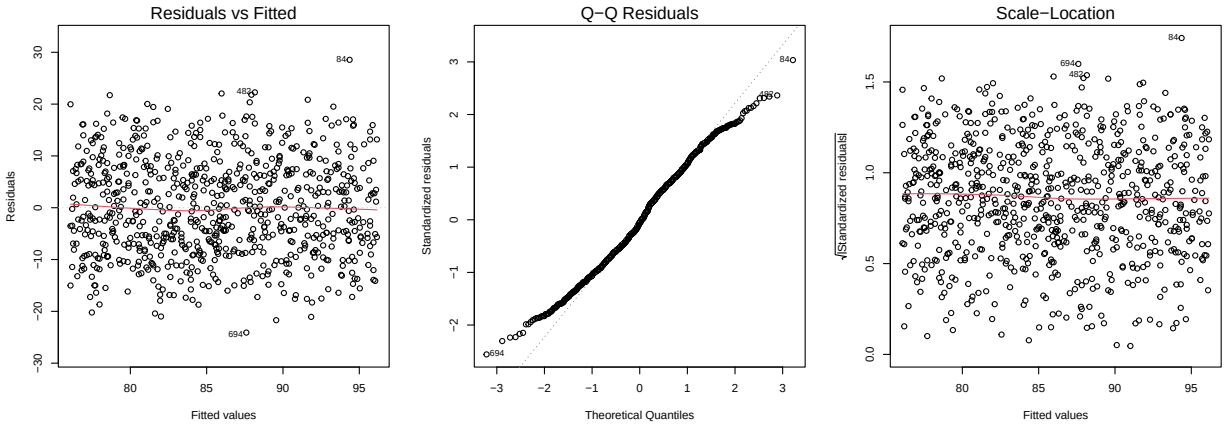
The R-squared value is approximately **0.270**, meaning that `relative_compactness` alone explains about **27.0%** of the variation in `heating_load`.

The residual standard error is approximately **9.428 kWh/m²**, so predictions from this model are typically about 9.428 kWh/m² away from the actual heating load values. This single-predictor model is useful as a baseline, but it is probably not adequate for design decisions because around 73% of the variation remains unexplained.

e) Diagnostic plots and model assumptions

```
par(mfrow = c(1, 3))

plot(simple_model, which = 1)
plot(simple_model, which = 2)
plot(simple_model, which = 3)
```



```r
par(mfrow = c(1, 1))
```

The residuals vs fitted plot shows some visible pattern rather than a completely random cloud, so the linearity assumption is not fully satisfied.

The Normal Q-Q plot shows that most residuals are reasonably close to the straight line, although there are some departures at the tails, so the normality assumption is broadly acceptable but not perfect.

The scale-location plot shows some variation in residual spread across fitted values, so the constant-variance assumption is not fully satisfied. Overall, the simple linear model is useful as a baseline, but it does not fully capture the structure in the data.

f) Polynomial regression for `glazing_area` and model selection

```r
poly_1 <- lm(heating_load ~ poly(glazing_area, 1, raw = TRUE), data = energy)
poly_2 <- lm(heating_load ~ poly(glazing_area, 2, raw = TRUE), data = energy)
poly_3 <- lm(heating_load ~ poly(glazing_area, 3, raw = TRUE), data = energy)
poly_4 <- lm(heating_load ~ poly(glazing_area, 4, raw = TRUE), data = energy)
poly_5 <- lm(heating_load ~ poly(glazing_area, 5, raw = TRUE), data = energy)

poly_results <- data.frame(
  Order = c(1, 2, 3, 4, 5),
  Adjusted_R_Squared = c(
    summary(poly_1)$adj.r.squared,
    summary(poly_2)$adj.r.squared,
    summary(poly_3)$adj.r.squared,
    summary(poly_4)$adj.r.squared,
    summary(poly_5)$adj.r.squared
  ),
  AIC = c(
    AIC(poly_1),
    AIC(poly_2),
    AIC(poly_3),
    AIC(poly_4),
    AIC(poly_5)
  )
)

poly_results
```

```
##   Order Adjusted_R_Squared      AIC
## 1     1         0.02074393 5854.801
## 2     2         0.05794366 5826.055
## 3     3         0.05673652 5828.034
## 4     4         0.05673652 5828.034
## 5     5         0.05673652 5828.034
```

Polynomial regression models of orders 1 to 5 were fitted to examine whether the relationship between `glazing_area` and `heating_load` is better represented by a curved relationship rather than a straight line. Adjusted R-squared and AIC are used because they consider both model fit and model complexity.

Based on the polynomial model comparison, the order 2 polynomial is preferred. The adjusted R-squared increases from approximately **0.021** for order 1 to approximately **0.058** for order 2, and AIC decreases from approximately **5854.8** to approximately **5826.1**. Higher-order models do not provide a meaningful improvement, because orders 3, 4, and 5 have slightly lower adjusted R-squared and higher AIC than the order 2 model. Therefore, order 2 gives the best balance between fit and unnecessary complexity.

1.3 Multiple Regression and Interaction

a) Full multiple regression and reduced Model A

```
full_model <- lm(
  heating_load ~ relative_compactness + surface_area + wall_area +
    roof_area + overall_height + orientation + glazing_area +
    glazing_distribution + wall_insul_rating + window_age,
  data = energy
)

summary(full_model)
```

```
##
## Call:
## lm(formula = heating_load ~ relative_compactness + surface_area +
##     wall_area + roof_area + overall_height + orientation + glazing_area +
##     glazing_distribution + wall_insul_rating + window_age, data = energy)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.9774 -2.0117  0.0382  2.3967 10.1198
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)            71.467665   1.737730  41.127   <2e-16 ***
## relative_compactness -57.096442   1.160166 -49.214   <2e-16 ***
## surface_area           0.044679   0.001378  32.417   <2e-16 ***
## wall_area              0.044501   0.002378  18.717   <2e-16 ***
## roof_area             -0.008562   0.003827  -2.237   0.0256 *
## overall_height         4.096988   0.067709  60.509   <2e-16 ***
## orientation            0.219941   0.104918   2.096   0.0364 *
## glazing_area           9.368598   0.899689  10.413   <2e-16 ***
## glazing_distribution   0.094675   0.070109   1.350   0.1773
## wall_insul_rating     -2.105365   0.101104 -20.824   <2e-16 ***
## window_age            -0.002423   0.013502  -0.179   0.8577
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.27 on 757 degrees of freedom
## Multiple R-squared:  0.9132, Adjusted R-squared:  0.9121
## F-statistic: 796.8 on 10 and 757 DF,  p-value: < 2.2e-16
```

The full multiple regression model uses all ten building-design predictors to explain `heating_load`. Unlike the simple linear regression model, this model estimates the relationship between each predictor and `heating_load` while holding the other predictors constant.

```
full_pvalues <- summary(full_model)$coefficients[, "Pr(>|t|)"]

full_pvalues
```

```
##          (Intercept) relative_compactness         surface_area
##        3.854219e-195        4.374037e-238        3.042191e-145
```

```
##            wall_area            roof_area       overall_height
##         1.544983e-64         2.555215e-02        3.245441e-292
##          orientation         glazing_area glazing_distribution
##         3.638640e-02         8.019488e-24         1.772936e-01
##    wall_insul_rating          window_age
##         1.728081e-76         8.576515e-01
```

```
insignificant_predictors <- names(full_pvalues[full_pvalues >= 0.05])
insignificant_predictors <- insignificant_predictors[insignificant_predictors != "(Intercept)"]

insignificant_predictors
```

```
## [1] "glazing_distribution" "window_age"
```

In the full model, the predictors with p-values greater than or equal to 0.05 are `glazing_distribution` and `window_age`. These predictors are treated as statistically insignificant at the 5% level and are removed from the reduced model. The variable `orientation` is retained because its p-value is below 0.05 in the full model.

```
model_A <- lm(
  heating_load ~ relative_compactness + surface_area + wall_area +
    roof_area + overall_height + orientation + glazing_area +
    wall_insul_rating,
  data = energy
)

summary(model_A)
```

```
##
## Call:
## lm(formula = heating_load ~ relative_compactness + surface_area +
##     wall_area + roof_area + overall_height + orientation + glazing_area +
##     wall_insul_rating, data = energy)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -9.1626 -2.0112  0.0072  2.3877 10.1878
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)            71.645352   1.725025  41.533   <2e-16 ***
## relative_compactness -56.972955   1.156244 -49.274   <2e-16 ***
## surface_area           0.044560   0.001375  32.412   <2e-16 ***
## wall_area              0.044628   0.002375  18.787   <2e-16 ***
## roof_area             -0.008613   0.003826  -2.251   0.0247 *
## overall_height         4.095204   0.067642  60.542   <2e-16 ***
## orientation            0.214061   0.104817   2.042   0.0415 *
## glazing_area           9.348640   0.899330  10.395   <2e-16 ***
## wall_insul_rating     -2.103975   0.101088 -20.813   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.27 on 759 degrees of freedom
## Multiple R-squared:  0.913,  Adjusted R-squared:  0.9121
## F-statistic:   996 on 8 and 759 DF,  p-value: < 2.2e-16
```

Model A is the reduced multiple regression model. It removes `glazing_distribution` and `window_age` from the full model while keeping the predictors that are statistically significant at the 5% level. Model A is therefore simpler than the full model but still keeps the main building-design variables that help explain `heating_load`.

b) Interaction Model B and interpretation

```
model_B <- lm(
  heating_load ~ relative_compactness + surface_area + wall_area +
    roof_area + overall_height + orientation + glazing_area +
    wall_insul_rating + wall_area:glazing_area,
  data = energy
)

summary(model_B)
```

```
##
## Call:
## lm(formula = heating_load ~ relative_compactness + surface_area +
##     wall_area + roof_area + overall_height + orientation + glazing_area +
##     wall_insul_rating + wall_area:glazing_area, data = energy)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.4830 -2.1080  0.0244  2.2848 10.2930
##
## Coefficients:
##                           Estimate Std. Error t value Pr(>|t|)
## (Intercept)             75.077337   2.078315  36.124  < 2e-16 ***
## relative_compactness   -57.108615   1.151447 -49.597  < 2e-16 ***
## surface_area             0.044667   0.001368  32.640  < 2e-16 ***
## wall_area                0.034097   0.004303   7.924 8.18e-15 ***
## roof_area               -0.008764   0.003807  -2.302   0.0216 *
## overall_height           4.109569   0.067485  60.896  < 2e-16 ***
## orientation              0.192227   0.104563   1.838   0.0664 .
## glazing_area            -7.354187   5.772682  -1.274   0.2031
## wall_insul_rating       -2.083308   0.100834 -20.661  < 2e-16 ***
## wall_area:glazing_area   0.050985   0.017408   2.929   0.0035 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.254 on 758 degrees of freedom
## Multiple R-squared:  0.914,  Adjusted R-squared:  0.913
## F-statistic: 895.2 on 9 and 758 DF,  p-value: < 2.2e-16
```

Model B extends Model A by adding an interaction term between `wall_area` and `glazing_area`. This interaction tests whether the effect of `glazing_area` on `heating_load` changes depending on the value of `wall_area`.

```
summary(model_B)$coefficients["wall_area:glazing_area", ]
```

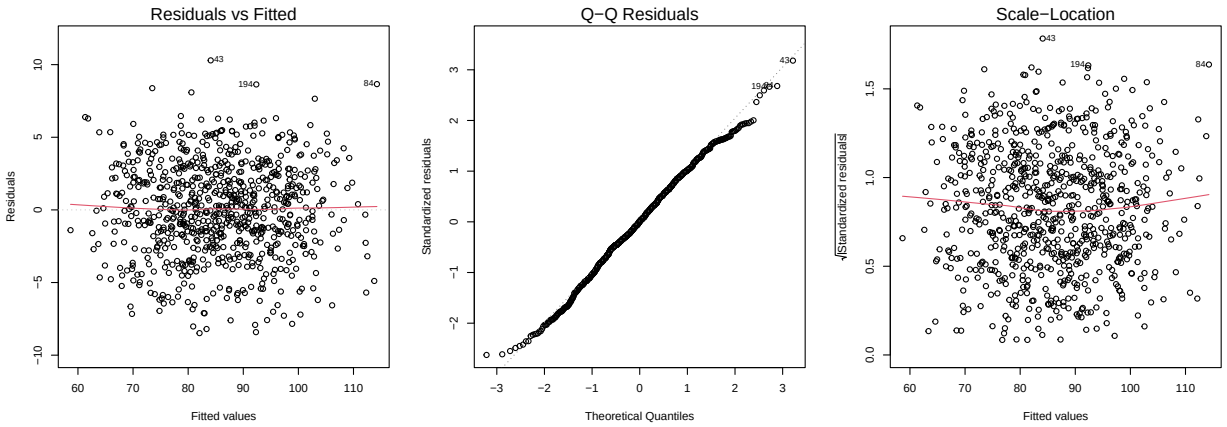
```
##   Estimate Std. Error    t value   Pr(>|t|)
## 0.05098505 0.01740799 2.92883078 0.00350444
```

The interaction term between `wall_area` and `glazing_area` is statistically significant, with a p-value of approximately **0.0035**. The interaction coefficient is positive, approximately **0.051**, meaning that the effect of `glazing_area` on `heating_load` becomes stronger as `wall_area` increases. In practical building-design terms, additional glazing is associated with a different heating-load effect for buildings with larger wall areas compared with buildings with smaller wall areas.

c) Diagnostic checks for Model B

```
par(mfrow = c(1, 3))

plot(model_B, which = 1)
plot(model_B, which = 2)
plot(model_B, which = 3)
```




```
par(mfrow = c(1, 1))
```

The residuals vs fitted plot for Model B shows residuals mostly centred around zero, although there is still some visible structure. This suggests that the linearity assumption is reasonably acceptable but not perfect.

The Normal Q-Q plot shows that most residuals follow the straight line, with some departures at the tails. This suggests that the normality assumption is broadly acceptable but not perfect.

The scale-location plot shows a reasonably even spread of residuals across fitted values, although some variation remains. Overall, the diagnostic plots suggest that Model B is acceptable for this assignment, but the regression assumptions are not perfectly satisfied.

d) Model A versus Model B recommendation

```
model_comparison <- data.frame(
  Model = c("Model A", "Model B"),
  Adjusted_R_Squared = c(
    summary(model_A)$adj.r.squared,
    summary(model_B)$adj.r.squared
  ),
  Residual_Standard_Error = c(
    summary(model_A)$sigma,
    summary(model_B)$sigma
  ),
  AIC = c(
```

```
    AIC(model_A),
    AIC(model_B)
  )
)

model_comparison
```

```
##     Model Adjusted_R_Squared Residual_Standard_Error      AIC
## 1 Model A          0.9121140                3.270040 4010.293
## 2 Model B          0.9129828                3.253837 4003.650
```

Model A and Model B are compared using adjusted R-squared, residual standard error, and AIC. Adjusted R-squared is preferred over ordinary R-squared because it accounts for the number of predictors. Residual standard error measures the typical prediction error in kWh/m². AIC compares model fit while penalising unnecessary complexity.

Based on the comparison, Model B is recommended. Model B has a higher adjusted R-squared, a lower residual standard error, and a lower AIC than Model A. This means Model B provides a slightly better balance between model fit and complexity. For a project manager, Model B is preferable because it improves prediction performance while also capturing the practical idea that the effect of glazing area depends on wall area.

1.4 Honest Model Comparison by Cross-Validation

a) 10-fold CV MSE for Model A and Model B

```
model_A_glm <- glm(
  formula(model_A),
  data = energy,
  family = gaussian
)

model_B_glm <- glm(
  formula(model_B),
  data = energy,
  family = gaussian
)

set.seed(D)
cv_model_A <- cv.glm(
  data = energy,
  glmfit = model_A_glm,
  K = 10
)

set.seed(D)
cv_model_B <- cv.glm(
  data = energy,
  glmfit = model_B_glm,
  K = 10
)
```

```r
cv_model_results <- data.frame(
  Model = c("Model A", "Model B"),
  CV_MSE = c(
    cv_model_A$delta[1],
    cv_model_B$delta[1]
  )
)

cv_model_results
```

```
##     Model   CV_MSE
## 1 Model A 10.84055
## 2 Model B 10.75440
```

The 10-fold cross-validation results compare Model A and Model B using cross-validated mean squared error. A lower CV MSE indicates better estimated prediction performance on unseen data. The seed value `D = 599` is used before each cross-validation run so that the fold allocation is reproducible.

Model B has a slightly lower CV MSE than Model A, with **10.754** compared with **10.841**. Therefore, Model B is preferred from a prediction-performance perspective. This agrees with the earlier adjusted R-squared, residual standard error, and AIC comparison, which also favoured Model B. The improvement is small, but Model B is still the better model because it gives the lower estimated out-of-sample prediction error.

b) 10-fold CV comparison for polynomial orders 1 to 5

```r
poly_1_glm <- glm(
  formula(poly_1),
  data = energy,
  family = gaussian
)

poly_2_glm <- glm(
  formula(poly_2),
  data = energy,
  family = gaussian
)

poly_3_glm <- glm(
  formula(poly_3),
  data = energy,
  family = gaussian
)

poly_4_glm <- glm(
  formula(poly_4),
  data = energy,
  family = gaussian
)

poly_5_glm <- glm(
  formula(poly_5),
  data = energy,
  family = gaussian
)
```

```r
set.seed(D)
cv_poly_1 <- cv.glm(
  data = energy,
  glmfit = poly_1_glm,
  K = 10
)

set.seed(D)
cv_poly_2 <- cv.glm(
  data = energy,
  glmfit = poly_2_glm,
  K = 10
)

set.seed(D)
cv_poly_3 <- cv.glm(
  data = energy,
  glmfit = poly_3_glm,
  K = 10
)

set.seed(D)
cv_poly_4 <- cv.glm(
  data = energy,
  glmfit = poly_4_glm,
  K = 10
)

set.seed(D)
cv_poly_5 <- cv.glm(
  data = energy,
  glmfit = poly_5_glm,
  K = 10
)

poly_cv_results <- data.frame(
  Order = c(1, 2, 3, 4, 5),
  CV_MSE = c(
    cv_poly_1$delta[1],
    cv_poly_2$delta[1],
    cv_poly_3$delta[1],
    cv_poly_4$delta[1],
    cv_poly_5$delta[1]
  ),
  Adjusted_R_Squared = c(
    summary(poly_1)$adj.r.squared,
    summary(poly_2)$adj.r.squared,
    summary(poly_3)$adj.r.squared,
    summary(poly_4)$adj.r.squared,
    summary(poly_5)$adj.r.squared
  )
)
```

```r
poly_cv_results
```

```
##   Order   CV_MSE Adjusted_R_Squared
## 1     1 119.4469         0.02074393
## 2     2 114.9313         0.05794366
## 3     3 115.2163         0.05673652
## 4     4 115.2163         0.05673652
## 5     5 115.2163         0.05673652
```

```r
best_poly_order <- poly_cv_results$Order[
  which.min(poly_cv_results$CV_MSE)
]

best_poly_order
```

```
## [1] 2
```

The polynomial models are compared again using 10-fold cross-validated MSE. This gives an out-of-sample comparison of polynomial orders 1 to 5, rather than relying only on adjusted R-squared from the training data.

The lowest CV MSE is achieved by the order 2 polynomial model, with a CV MSE of approximately **114.931**. The order 1 model has a higher CV MSE of approximately **119.447**, while orders 3, 4, and 5 have CV MSE values of approximately **115.216**. Therefore, the 10-fold cross-validation result supports the same conclusion as Section 1.2: the order 2 polynomial provides the best balance for modelling the relationship between `glazing_area` and `heating_load`.

These polynomial CV MSE values should be compared only among the polynomial models, because these models use only `glazing_area` as the predictor. They are not directly comparable with Model A and Model B, which use multiple predictors.

c) Why cross-validated MSE is more trustworthy than training R^2

Training R-squared measures how well a model fits the same data used to estimate its coefficients. Because of this, training R-squared can be overly optimistic, especially when a model becomes more complex. Adding more predictors or higher-order polynomial terms can improve training fit even when those extra terms do not improve prediction on new data.

Cross-validated MSE is more trustworthy for model comparison because it evaluates prediction error on held-out folds. In 10-fold cross-validation, the data are split into ten parts. The model is repeatedly trained on nine parts and tested on the remaining part. This process gives a better estimate of how the model is likely to perform on unseen observations.

In this report, cross-validation supports the same choices as the earlier training-fit comparisons: Model B has a lower CV MSE than Model A (**10.754 versus 10.841**), and the order 2 polynomial has the lowest CV MSE among orders 1 to 5 (**114.931**). This strengthens the recommendation because the selected models also perform better under held-out validation.

2. Question 2 — Classification

This question uses `learners.csv` to predict whether a learner completed the course. Because the response variable `completed` is categorical, this is a classification task rather than a regression task.

2.1 Cleaning and Splitting

a) Missing values and cleaning decision

```
learners <- read.csv("D:/data-portfolio/07_learner_retention_analytics_r/data/learners.csv", stringsAsFa

dim(learners)
```

```
## [1] 2000   18
```

```
names(learners)
```

```
##  [1] "learner_id"        "age"               "prior_score"
##  [4] "prior_courses"     "weekly_hours"      "videos_watched"
##  [7] "assignments_done"  "quiz_avg"          "forum_posts"
## [10] "logins_per_week"   "support_tickets"   "days_since_signup"
## [13] "signup_hour"       "plan"              "device"
## [16] "region"            "email_verified"    "completed"
```

```
str(learners)
```

```
## 'data.frame':    2000 obs. of  18 variables:
##  $ learner_id       : chr  "L10000" "L10001" "L10002" "L10003" ...
##  $ age              : int  32 27 51 30 56 52 47 36 19 49 ...
##  $ prior_score      : num  39.6 75.2 68.6 63.4 59.5 40.4 68.2 68.4 65.4 89.5 ...
##  $ prior_courses    : int  4 5 0 1 1 2 0 5 2 2 ...
##  $ weekly_hours     : num  7.57 7.09 11.02 9 7.75 ...
##  $ videos_watched   : int  17 21 21 17 24 15 14 19 22 19 ...
##  $ assignments_done : int  6 3 5 9 1 2 0 7 5 10 ...
##  $ quiz_avg         : num  59.6 50.4 81.2 51.3 77.1 55.1 40.1 89.2 57.4 69.1 ...
##  $ forum_posts      : int  2 2 2 4 3 3 5 2 3 4 ...
##  $ logins_per_week  : num  4.7 2.8 3.7 4.6 2.2 6.5 3.5 3.9 8.3 2.9 ...
##  $ support_tickets  : int  1 1 2 2 0 0 0 1 3 0 ...
##  $ days_since_signup: int  27 116 94 67 5 38 28 56 31 31 ...
##  $ signup_hour      : int  11 21 16 9 16 10 14 16 14 20 ...
##  $ plan             : chr  "free" "basic" "free" "basic" ...
##  $ device           : chr  "mobile" "desktop" "desktop" "mobile" ...
##  $ region           : chr  "South" "South" "South" "North" ...
##  $ email_verified   : chr  "yes" "yes" "yes" "no" ...
##  $ completed        : chr  "no" "no" "no" "no" ...
```

```
head(learners)
```

```
##   learner_id age prior_score prior_courses weekly_hours videos_watched
## 1     L10000  32        39.6             4         7.57             17
## 2     L10001  27        75.2             5         7.09             21
## 3     L10002  51        68.6             0        11.02             21
## 4     L10003  30        63.4             1         9.00             17
## 5     L10004  56        59.5             1         7.75             24
## 6     L10005  52        40.4             2         4.01             15
##   assignments_done quiz_avg forum_posts logins_per_week support_tickets
```

```
## 1                6     59.6           2             4.7              1
## 2                3     50.4           2             2.8              1
## 3                5     81.2           2             3.7              2
## 4                9     51.3           4             4.6              2
## 5                1     77.1           3             2.2              0
## 6                2     55.1           3             6.5              0
##   days_since_signup signup_hour  plan  device region email_verified completed
## 1                27          11  free  mobile  South            yes        no
## 2               116          21 basic desktop  South            yes        no
## 3                94          16  free desktop  South            yes        no
## 4                67           9 basic  mobile  North             no        no
## 5                 5          16  free desktop  North            yes       yes
## 6                38          10  free  mobile  North            yes        no
```

```
for (col in names(learners)) {
  if (is.character(learners[[col]])) {
    learners[[col]][learners[[col]] == ""] <- NA
  }
}

colSums(is.na(learners))
```

```
##        learner_id               age       prior_score     prior_courses
##                 0                 0                 0                 0
##      weekly_hours    videos_watched  assignments_done          quiz_avg
##                 0                 0                 0                12
##       forum_posts   logins_per_week   support_tickets days_since_signup
##                 0                 8                 0                 0
##       signup_hour              plan            device            region
##                 0                 0                 5                 0
##    email_verified         completed
##                 0                 0
```

```
sum(is.na(learners))
```

```
## [1] 25
```

The `learners.csv` dataset contains 2000 rows and 18 columns. Blank text values were treated as missing values before checking missingness.

```
learners_clean <- learners[!is.na(learners$completed), ]

get_mode <- function(x) {
  x <- x[!is.na(x)]
  names(sort(table(x), decreasing = TRUE))[1]
}

for (col in names(learners_clean)) {
  if (col != "completed") {
    if (is.numeric(learners_clean[[col]])) {
      learners_clean[[col]][is.na(learners_clean[[col]])] <- median(
        learners_clean[[col]],
```

```
        na.rm = TRUE
      )
    } else {
      mode_value <- get_mode(learners_clean[[col]])
      learners_clean[[col]][is.na(learners_clean[[col]])] <- mode_value
    }
  }
}

colSums(is.na(learners_clean))
```

```
##        learner_id               age       prior_score     prior_courses
##                 0                 0                 0                 0
##      weekly_hours    videos_watched  assignments_done          quiz_avg
##                 0                 0                 0                 0
##       forum_posts   logins_per_week   support_tickets days_since_signup
##                 0                 0                 0                 0
##       signup_hour              plan            device            region
##                 0                 0                 0                 0
##    email_verified         completed
##                 0                 0
```

```
sum(is.na(learners_clean))
```

```
## [1] 0
```

```
dim(learners_clean)
```

```
## [1] 2000   18
```

Rows with missing `completed` values are removed because the response variable should not be imputed in a supervised classification task. For predictor variables, missing numeric values are replaced with the median, and missing categorical values are replaced with the most common category. This keeps as many usable learner records as possible while avoiding missing values in the modelling stage.

b) Factor conversion, completion rate, and class balance

```
for (col in names(learners_clean)) {
  if (is.character(learners_clean[[col]]) && col != "learner_id") {
    learners_clean[[col]] <- factor(learners_clean[[col]])
  }
}

learners_clean$completed <- factor(
  learners_clean$completed,
  levels = c("no", "yes")
)

q2_response <- "completed"

q2_predictors <- names(learners_clean)[
  !(names(learners_clean) %in% c("learner_id", "completed"))
```

```
]

str(learners_clean)
```

```
## 'data.frame':    2000 obs. of  18 variables:
##  $ learner_id       : chr  "L10000" "L10001" "L10002" "L10003" ...
##  $ age              : num  32 27 51 30 56 52 47 36 19 49 ...
##  $ prior_score      : num  39.6 75.2 68.6 63.4 59.5 40.4 68.2 68.4 65.4 89.5 ...
##  $ prior_courses    : num  4 5 0 1 1 2 0 5 2 2 ...
##  $ weekly_hours     : num  7.57 7.09 11.02 9 7.75 ...
##  $ videos_watched   : num  17 21 21 17 24 15 14 19 22 19 ...
##  $ assignments_done : num  6 3 5 9 1 2 0 7 5 10 ...
##  $ quiz_avg         : num  59.6 50.4 81.2 51.3 77.1 55.1 40.1 89.2 57.4 69.1 ...
##  $ forum_posts      : num  2 2 2 4 3 3 5 2 3 4 ...
##  $ logins_per_week  : num  4.7 2.8 3.7 4.6 2.2 6.5 3.5 3.9 8.3 2.9 ...
##  $ support_tickets  : num  1 1 2 2 0 0 0 1 3 0 ...
##  $ days_since_signup: num  27 116 94 67 5 38 28 56 31 31 ...
##  $ signup_hour      : num  11 21 16 9 16 10 14 16 14 20 ...
##  $ plan             : Factor w/ 3 levels "basic","free",..: 2 1 2 1 2 2 1 2 2 3 ...
##  $ device           : Factor w/ 3 levels "desktop","mobile",..: 2 1 1 2 1 2 2 3 1 1 ...
##  $ region           : Factor w/ 4 levels "East","North",..: 3 3 3 2 2 2 3 4 2 1 ...
##  $ email_verified   : Factor w/ 2 levels "no","yes": 2 2 2 1 2 2 2 2 2 2 ...
##  $ completed        : Factor w/ 2 levels "no","yes": 1 1 1 1 2 1 1 2 1 2 ...
```

```
levels(learners_clean$completed)
```

```
## [1] "no"  "yes"
```

```
q2_response
```

```
## [1] "completed"
```

```
q2_predictors
```

```
##  [1] "age"               "prior_score"       "prior_courses"
##  [4] "weekly_hours"      "videos_watched"    "assignments_done"
##  [7] "quiz_avg"          "forum_posts"       "logins_per_week"
## [10] "support_tickets"   "days_since_signup" "signup_hour"
## [13] "plan"              "device"            "region"
## [16] "email_verified"
```

```
completion_table <- table(learners_clean$completed)
completion_rate <- prop.table(completion_table)

completion_table
```

```
##
##   no  yes
## 1143  857
```

```r
round(completion_rate, 3)
```

```
##
##    no   yes
## 0.572 0.428
```

Categorical predictor variables are converted to factors so that R treats them as categories rather than continuous numeric variables. The response variable `completed` is also converted to a factor with `no` as the first level and `yes` as the second level. The variable `learner_id` is kept as an identifier and is not used as a predictor in the classification model.

The completion table and completion-rate output show the class distribution of the response variable. This check is important because class imbalance can make accuracy misleading. In this dataset, the two classes are reasonably balanced rather than extremely imbalanced, so accuracy is useful, but it should still be reported together with sensitivity, specificity, and AUC in the model evaluation stage.

c) Personalised 80/20 train-test split

```r
set.seed(D)

train_rows <- c()

for (class_level in levels(learners_clean$completed)) {
  class_rows <- which(learners_clean$completed == class_level)

  class_train_rows <- sample(
    class_rows,
    size = floor(0.8 * length(class_rows))
  )

  train_rows <- c(train_rows, class_train_rows)
}

train_rows <- sort(train_rows)

learners_train <- learners_clean[train_rows, ]
learners_test <- learners_clean[-train_rows, ]

dim(learners_train)
```

```
## [1] 1599   18
```

```r
dim(learners_test)
```

```
## [1] 401  18
```

```r
table(learners_train$completed)
```

```
##
##  no yes
## 914 685
```

```r
round(prop.table(table(learners_train$completed)), 3)
```

```
##
##    no   yes
## 0.572 0.428
```

```r
table(learners_test$completed)
```

```
##
##  no yes
## 229 172
```

```r
round(prop.table(table(learners_test$completed)), 3)
```

```
##
##    no   yes
## 0.571 0.429
```

The dataset is split into an 80% training set and a 20% test set using the personalised seed value `D = 599`. The split is stratified by `completed`, meaning that each completion class is sampled separately. This helps keep the class distribution in the training and test sets similar to the original dataset.

The training set is used to fit the logistic regression model, while the test set is held out for model evaluation. This avoids evaluating the model only on the same data used for training.

2.2 Logistic Regression and Test Set Evaluation

a) Full logistic regression model

```r
full_logit <- glm(
  completed ~ age + prior_score + prior_courses + weekly_hours +
    videos_watched + assignments_done + quiz_avg + forum_posts +
    logins_per_week + support_tickets + days_since_signup +
    signup_hour + plan + device + region + email_verified,
  data = learners_train,
  family = binomial
)

summary(full_logit)
```

```
##
## Call:
## glm(formula = completed ~ age + prior_score + prior_courses +
##     weekly_hours + videos_watched + assignments_done + quiz_avg +
##     forum_posts + logins_per_week + support_tickets + days_since_signup +
##     signup_hour + plan + device + region + email_verified, family = binomial,
##     data = learners_train)
##
## Coefficients:
##                     Estimate Std. Error z value Pr(>|z|)
```

```
## (Intercept)       -6.1797320  0.5954132 -10.379  < 2e-16 ***
## age               -0.0024746  0.0048604  -0.509  0.61065
## prior_score        0.0317373  0.0041230   7.698 1.39e-14 ***
## prior_courses      0.1312122  0.0341068   3.847  0.00012 ***
## weekly_hours       0.1470345  0.0201384   7.301 2.85e-13 ***
## videos_watched     0.0377519  0.0134061   2.816  0.00486 **
## assignments_done   0.2818573  0.0198081  14.229  < 2e-16 ***
## quiz_avg           0.0177223  0.0033374   5.310 1.09e-07 ***
## forum_posts       -0.0051520  0.0341622  -0.151  0.88012
## logins_per_week    0.0656317  0.0302296   2.171  0.02992 *
## support_tickets   -0.0698849  0.0682811  -1.023  0.30608
## days_since_signup -0.0011569  0.0017455  -0.663  0.50746
## signup_hour       -0.0005324  0.0084201  -0.063  0.94958
## planfree          -0.6422184  0.1351746  -4.751 2.02e-06 ***
## planpremium        0.2578286  0.1636577   1.575  0.11516
## devicemobile      -0.2719443  0.1243920  -2.186  0.02880 *
## devicetablet      -0.1927506  0.2059134  -0.936  0.34923
## regionNorth        0.0963903  0.1630393   0.591  0.55438
## regionSouth       -0.1049432  0.1722534  -0.609  0.54237
## regionWest        -0.1137519  0.1678771  -0.678  0.49803
## email_verifiedyes  0.2377398  0.1597283   1.488  0.13665
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 2183.8  on 1598  degrees of freedom
## Residual deviance: 1753.8  on 1578  degrees of freedom
## AIC: 1795.8
##
## Number of Fisher Scoring iterations: 4
```

The full logistic regression model uses all learner-behaviour, background, and account-related predictors to predict whether a learner completed the course. The identifier variable `learner_id` is not included because it is only a learner ID and does not provide meaningful predictive information.

```
full_logit_pvalues <- summary(full_logit)$coefficients[, "Pr(>|z|)"]

full_logit_pvalues
```

```
##       (Intercept)               age       prior_score     prior_courses
##      3.093316e-25      6.106519e-01      1.386406e-14      1.195230e-04
##      weekly_hours    videos_watched  assignments_done          quiz_avg
##      2.851785e-13      4.862264e-03      6.017484e-46      1.094668e-07
##       forum_posts   logins_per_week   support_tickets days_since_signup
##      8.801246e-01      2.992330e-02      3.060770e-01      5.074648e-01
##       signup_hour          planfree       planpremium      devicemobile
##      9.495800e-01      2.023843e-06      1.151608e-01      2.880191e-02
##      devicetablet       regionNorth       regionSouth        regionWest
##      3.492341e-01      5.543804e-01      5.423672e-01      4.980315e-01
## email_verifiedyes
##      1.366451e-01
```

At the 5% significance level, the full logistic regression model suggests that several learner-engagement and performance variables are useful predictors of course completion. These include `prior_score`, `prior_courses`, `weekly_hours`, `videos_watched`, `assignments_done`, `quiz_avg`, and `logins_per_week`. Some categorical levels, such as `planfree` and `devicemobile`, also show statistical evidence of association with course completion.

By contrast, several variables have weaker statistical evidence in the full model, including `age`, `forum_posts`, `support_tickets`, `days_since_signup`, `signup_hour`, most `region` levels, and `email_verified`. These weaker predictors are not kept in the refined final model.

b) Refined final logistic model

```
final_logit <- glm(
  completed ~ prior_score + prior_courses + weekly_hours +
    videos_watched + assignments_done + quiz_avg +
    logins_per_week + plan + device,
  data = learners_train,
  family = binomial
)

summary(final_logit)
```

```
##
## Call:
## glm(formula = completed ~ prior_score + prior_courses + weekly_hours +
##     videos_watched + assignments_done + quiz_avg + logins_per_week +
##     plan + device, family = binomial, data = learners_train)
##
## Coefficients:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)      -6.232175   0.508657 -12.252  < 2e-16 ***
## prior_score       0.031615   0.004109   7.694 1.42e-14 ***
## prior_courses     0.134529   0.033872   3.972 7.14e-05 ***
## weekly_hours      0.146303   0.020033   7.303 2.81e-13 ***
## videos_watched    0.038593   0.013358   2.889  0.00386 **
## assignments_done  0.279901   0.019680  14.222  < 2e-16 ***
## quiz_avg          0.017757   0.003316   5.355 8.56e-08 ***
## logins_per_week   0.061764   0.030011   2.058  0.03959 *
## planfree         -0.645614   0.134604  -4.796 1.62e-06 ***
## planpremium       0.246289   0.162905   1.512  0.13057
## devicemobile     -0.267541   0.123262  -2.171  0.02997 *
## devicetablet     -0.204492   0.205481  -0.995  0.31965
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 2183.8  on 1598  degrees of freedom
## Residual deviance: 1759.8  on 1587  degrees of freedom
## AIC: 1783.8
##
## Number of Fisher Scoring iterations: 4
```

The refined final logistic regression model keeps the main predictors with stronger evidence from the full model. It includes learner performance variables, engagement variables, plan type, and device type. Predic-

tors with weaker evidence in the full model are removed to make the final model easier to interpret and less unnecessarily complex.

```
summary(final_logit)$coefficients
```

```
##                     Estimate  Std. Error     z value     Pr(>|z|)
## (Intercept)      -6.23217485 0.508656692 -12.2522223 1.634546e-34
## prior_score       0.03161511 0.004108946   7.6942142 1.423663e-14
## prior_courses     0.13452917 0.033871732   3.9717239 7.135439e-05
## weekly_hours      0.14630276 0.020032951   7.3031055 2.812004e-13
## videos_watched    0.03859319 0.013358441   2.8890488 3.864091e-03
## assignments_done  0.27990114 0.019680216  14.2224628 6.647178e-46
## quiz_avg          0.01775660 0.003315948   5.3549104 8.559882e-08
## logins_per_week   0.06176436 0.030011288   2.0580377 3.958652e-02
## planfree         -0.64561428 0.134603916  -4.7964004 1.615422e-06
## planpremium       0.24628904 0.162905424   1.5118529 1.305713e-01
## devicemobile     -0.26754087 0.123261898  -2.1705075 2.996842e-02
## devicetablet     -0.20449207 0.205481236  -0.9951861 3.196458e-01
```

The final model is used for test-set prediction and evaluation. In logistic regression, coefficients are estimated on the log-odds scale, so odds ratios are also calculated to make the interpretation easier.

c) Odds ratio interpretation

```
odds_ratios <- exp(coef(final_logit))

odds_ratios
```

```
##      (Intercept)      prior_score    prior_courses     weekly_hours
##      0.001965173      1.032120178      1.143998026      1.157546588
##   videos_watched assignments_done         quiz_avg  logins_per_week
##      1.039347578      1.322999012      1.017915189      1.063711663
##         planfree      planpremium     devicemobile     devicetablet
##      0.524340353      1.279269277      0.765259051      0.815061208
```

An odds ratio greater than 1 means that higher values of the predictor are associated with higher odds of course completion, holding the other variables constant. An odds ratio below 1 means that higher values of the predictor are associated with lower odds of course completion, holding the other variables constant. An odds ratio is not the same as a probability ratio; it describes changes in odds, not direct changes in probability.

```
odds_ratios["assignments_done"]
```

```
## assignments_done
##         1.322999
```

```
odds_ratios["devicemobile"]
```

```
## devicemobile
##    0.7652591
```

For the numeric predictor `assignments_done`, the odds ratio is approximately **1.323**. Holding the other predictors constant, each additional completed assignment multiplies the odds of course completion by about **1.323**. In other words, one extra completed assignment is associated with approximately **32.3% higher odds** of completion.

For the categorical predictor `devicemobile`, the odds ratio is approximately **0.765**. Since `desktop` is the reference device group, this means that, holding the other predictors constant, mobile-device learners have about **0.765 times the odds** of completion compared with desktop-device learners. This is approximately **23.5% lower odds** of completion. These are odds ratios, not probability ratios, so they should not be interpreted as direct percentage changes in probability.

d) Test set prediction, confusion matrix, accuracy, sensitivity, and specificity

```
test_prob <- predict(
  final_logit,
  newdata = learners_test,
  type = "response"
)

test_pred <- ifelse(test_prob >= 0.5, "yes", "no")

test_pred <- factor(
  test_pred,
  levels = c("no", "yes")
)

confusion_matrix <- table(
  Actual = learners_test$completed,
  Predicted = test_pred
)

confusion_matrix
```

```
##       Predicted
## Actual  no yes
##    no  184  45
##    yes  61 111
```

```
tn <- confusion_matrix["no", "no"]
fp <- confusion_matrix["no", "yes"]
fn <- confusion_matrix["yes", "no"]
tp <- confusion_matrix["yes", "yes"]

accuracy <- (tp + tn) / sum(confusion_matrix)
sensitivity <- tp / (tp + fn)
specificity <- tn / (tn + fp)

accuracy
```

```
## [1] 0.7356608
```

```
sensitivity
```

```
## [1] 0.6453488
```

```
specificity
```

```
## [1] 0.8034934
```

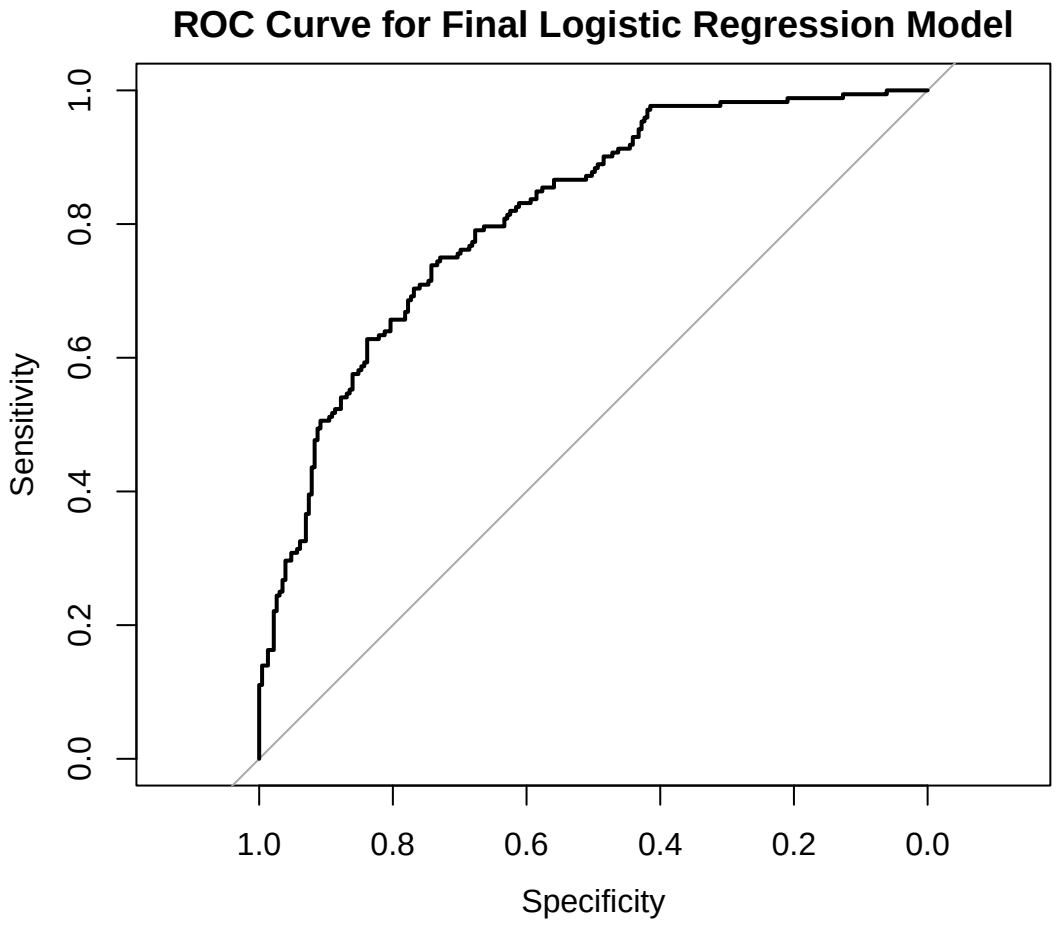
The test-set predictions are created using a probability threshold of 0.5. If the predicted probability of completion is at least 0.5, the learner is classified as `yes`; otherwise, the learner is classified as `no`.

Accuracy measures the overall proportion of correct predictions. Sensitivity measures the proportion of actual completers correctly identified as completers. Specificity measures the proportion of actual non-completers correctly identified as non-completers. Because the target classes are not extremely imbalanced, accuracy is useful, but sensitivity and specificity are also reported to show how well the model performs for each class.

e) ROC curve and AUC

```
roc_result <- roc(
  response = learners_test$completed,
  predictor = test_prob,
  levels = c("no", "yes")
)

plot(
  roc_result,
  main = "ROC Curve for Final Logistic Regression Model"
)
```



```r
auc_value <- auc(roc_result)

auc_value
```

```
## Area under the curve: 0.8128
```

The ROC curve evaluates the model across many possible probability thresholds, rather than only using the 0.5 threshold. The AUC value summarises the model's ability to distinguish learners who completed the course from learners who did not. A higher AUC indicates better discrimination. Therefore, AUC is useful because it evaluates ranking performance across thresholds, while accuracy only evaluates one chosen threshold.

Because the classes are reasonably balanced but not perfectly balanced (**57.2% no and 42.8% yes**), AUC is useful as a threshold-independent measure of discrimination, while accuracy alone may hide differences in how well the model handles each class.

f) Threshold adjustment for at-risk learner detection

```r
at_risk_threshold <- 0.60

test_pred_060 <- ifelse(test_prob >= at_risk_threshold, "yes", "no")

test_pred_060 <- factor(
  test_pred_060,
  levels = c("no", "yes")
)

confusion_matrix_060 <- table(
  Actual = learners_test$completed,
  Predicted = test_pred_060
)

confusion_matrix_060
```

```
##       Predicted
## Actual  no yes
##    no  201  28
##    yes  80  92
```

```r
tn_060 <- confusion_matrix_060["no", "no"]
fp_060 <- confusion_matrix_060["no", "yes"]
fn_060 <- confusion_matrix_060["yes", "no"]
tp_060 <- confusion_matrix_060["yes", "yes"]

accuracy_060 <- (tp_060 + tn_060) / sum(confusion_matrix_060)
sensitivity_060 <- tp_060 / (tp_060 + fn_060)
specificity_060 <- tn_060 / (tn_060 + fp_060)

accuracy_060
```

```
## [1] 0.7306733
```

```
sensitivity_060
```

```
## [1] 0.5348837
```

```
specificity_060
```

```
## [1] 0.8777293
```

The threshold is increased from 0.5 to 0.6 because the practical goal is to identify at-risk learners, meaning learners who are likely not to complete the course. Since the model predicts the probability of `completed = yes`, a higher threshold makes it harder for a learner to be classified as a completer. As a result, more learners are classified as `no`, which can improve the detection of non-completers.

This threshold adjustment creates a trade-off. It is expected to increase specificity, meaning that more actual non-completers are correctly identified. However, it may reduce sensitivity, meaning that fewer actual completers are correctly classified as completers. For at-risk learner detection, this trade-off can be acceptable because missing at-risk learners may be more costly than incorrectly flagging some learners for extra support.

2.3 Cross-Validated Assessment

a) 10-fold CV misclassification rate

```
classification_cost <- function(actual, predicted_probability) {
  predicted_class <- ifelse(predicted_probability >= 0.5, 1, 0)
  mean(actual != predicted_class)
}

final_logit_cv <- glm(
  formula(final_logit),
  data = learners_clean,
  family = binomial
)

set.seed(D)

cv_final_logit <- cv.glm(
  data = learners_clean,
  glmfit = final_logit_cv,
  cost = classification_cost,
  K = 10
)

cv_error <- cv_final_logit$delta[1]
cv_accuracy <- 1 - cv_error

cv_error
```

```
## [1] 0.282
```

```
cv_accuracy
```

```
## [1] 0.718
```

The 10-fold cross-validation uses the same final logistic regression formula as Section 2.2. The customised cost function converts predicted probabilities into predicted classes using the 0.5 threshold and then calculates the misclassification rate.

The cross-validated misclassification rate is approximately **0.282**, so the corresponding cross-validated accuracy is approximately **0.718**.

b) CV error versus single-split test error

```
test_error_050 <- 1 - accuracy

cv_test_comparison <- data.frame(
  Method = c("Single 80/20 test split", "10-fold cross-validation"),
  Misclassification_Rate = c(
    test_error_050,
    cv_error
  ),
  Accuracy = c(
    accuracy,
    cv_accuracy
  )
)

cv_test_comparison
```

```
##                     Method Misclassification_Rate  Accuracy
## 1  Single 80/20 test split              0.2643392 0.7356608
## 2 10-fold cross-validation              0.2820000 0.7180000
```

The single 80/20 test split gives a test-set misclassification rate of approximately **0.264**, while the 10-fold cross-validation gives a misclassification rate of approximately **0.282**. These two values are fairly similar, so the earlier train-test split does not appear to be extremely lucky or unlucky. However, the single split is slightly more optimistic because its error rate is lower than the cross-validated error.

A large gap between the single-split test error and the cross-validated error would suggest that the earlier split may not be representative. For example, a much lower single-split error would suggest a lucky test split, while a much higher single-split error would suggest an unusually difficult test split.

c) Why cross-validation is useful here

Cross-validation is useful in this classification problem because the dataset has 2,000 learners, which is large enough to split into ten folds while still leaving enough observations in each fold. The response classes are also reasonably balanced, with about 57.2% `no` and 42.8% `yes`, so each fold should contain both classes in useful numbers.

A single 80/20 split can still be slightly lucky or unlucky, even when it is stratified. In 10-fold cross-validation, the model is repeatedly trained on nine folds and tested on the remaining fold, so every observation is used for validation once. This gives a more stable estimate of the final logistic model's likely performance on unseen learners than relying only on one test split.

3. Question 3 — Synthesis and Reproducibility

3.1 Executive Summary

For the learning platform, the most useful finding is that course completion can be predicted with reasonably useful, but not perfect, accuracy from early engagement and performance variables. The final logistic model achieved a test accuracy of **0.736** at the 0.5 threshold and an AUC of **0.813**, meaning it can separate completers from non-completers fairly well across thresholds. For at-risk learner detection, raising the threshold to **0.60** increased specificity from **0.803** to **0.878**, so more actual non-completers were correctly identified. The main limitation is that this trade-off reduced sensitivity from **0.645** to **0.535**, meaning more actual completers would be incorrectly flagged as at risk. Therefore, the model is useful for prioritising support, but it should not be used as the only basis for intervention decisions.

3.2 Reproducibility Statement

This analysis can be reproduced by placing `energy_homes.csv` and `learners.csv` in the `data` folder, opening the same R Markdown file, installing the required packages, and running the report from top to bottom. The personalised seed value used in this report is `D = 599`, from the last three digits of Student ID `22145599`.

The command `set.seed(D)` affects every random procedure in the report. In Question 1, it affects the 10-fold cross-validation folds for comparing Model A and Model B, and the 10-fold cross-validation folds for the polynomial models of orders 1 to 5. In Question 2, it affects the stratified 80/20 train-test split, which then affects the fitted logistic regression model, the test-set confusion matrix, accuracy, sensitivity, specificity, ROC curve, AUC, and threshold-adjustment results. It also affects the 10-fold cross-validation folds used to estimate the final logistic model's misclassification rate.

If a different student ran the same code using their own value of `D`, the deterministic outputs would stay the same, including the missing-value counts, completion rate, and regression models fitted on the full `energy` dataset. However, outputs depending on random folds or the train-test split could change slightly, including the cross-validated MSE values, the selected training and test rows, the logistic regression estimates fitted on the training set, and the test-set performance metrics.

3.3 Regression versus Classification

Question 1 is a regression problem because the response variable, `heating_load`, is numeric and continuous. This means model errors can be measured as numeric distances between predicted and actual heating load values. Therefore, metrics such as MSE, residual standard error, and R-squared are appropriate because they measure prediction error and explained variation for a continuous outcome.

Question 2 is a classification problem because the response variable, `completed`, has two categories: `yes` and `no`. For this type of outcome, the main question is whether each learner is placed into the correct class, not how far a numeric prediction is from a numeric target. Therefore, a confusion matrix, accuracy, sensitivity, specificity, ROC curve, AUC, and misclassification rate are more appropriate. These metrics evaluate how well the model separates completers from non-completers and how its performance changes under different probability thresholds.

4. Bonus — Least Useful Predictor Experiment

The predictor selected for this bonus experiment is window_age. It was chosen because it had one of the weakest relationships with heating_load in the exploratory analysis and was statistically insignificant in the full regression model.

```r
# Model without window_age
bonus_without <- lm(
  heating_load ~ relative_compactness + surface_area + wall_area +
    roof_area + overall_height + orientation + glazing_area +
    glazing_distribution + wall_insul_rating,
  data = energy
)

# Model with window_age
bonus_with <- lm(
  heating_load ~ relative_compactness + surface_area + wall_area +
    roof_area + overall_height + orientation + glazing_area +
    glazing_distribution + wall_insul_rating + window_age,
  data = energy
)

# Check window_age coefficient
summary(bonus_with)$coefficients["window_age", ]
```

```
##     Estimate   Std. Error      t value     Pr(>|t|)
## -0.002422535  0.013501610 -0.179425619  0.857651530
```

```r
# Training comparison
bonus_training <- data.frame(
  Model = c("Without window_age", "With window_age"),
  Training_MSE = c(
    mean(residuals(bonus_without)^2),
    mean(residuals(bonus_with)^2)
  ),
  R_Squared = c(
    summary(bonus_without)$r.squared,
    summary(bonus_with)$r.squared
  ),
  Adjusted_R_Squared = c(
    summary(bonus_without)$adj.r.squared,
    summary(bonus_with)$adj.r.squared
  ),
  AIC = c(
    AIC(bonus_without),
    AIC(bonus_with)
  )
)

bonus_training
```

```
##                Model Training_MSE R_Squared Adjusted_R_Squared      AIC
## 1 Without window_age     10.54253 0.9132390          0.9122089 4010.450
## 2    With window_age     10.54208 0.9132427          0.9120967 4012.418
```

```r
# Use glm only for cv.glm
bonus_without_glm <- glm(
  heating_load ~ relative_compactness + surface_area + wall_area +
```

```
    roof_area + overall_height + orientation + glazing_area +
    glazing_distribution + wall_insul_rating,
  data = energy
)

bonus_with_glm <- glm(
  heating_load ~ relative_compactness + surface_area + wall_area +
    roof_area + overall_height + orientation + glazing_area +
    glazing_distribution + wall_insul_rating + window_age,
  data = energy
)

# 10-fold CV comparison
set.seed(D)
cv_without <- cv.glm(energy, bonus_without_glm, K = 10)$delta[1]

set.seed(D)
cv_with <- cv.glm(energy, bonus_with_glm, K = 10)$delta[1]

bonus_cv <- data.frame(
  Model = c("Without window_age", "With window_age"),
  CV_MSE = c(cv_without, cv_with)
)

bonus_cv
```

```
##                Model   CV_MSE
## 1 Without window_age 10.82843
## 2    With window_age 10.85146
```

The results confirm that window_age adds essentially no useful predictive value.

The coefficient of `window_age` is approximately -0.0024, with a p-value of 0.858. Holding other predictors constant, a one-year increase in window age is associated with only a 0.0024 kWh/m² decrease in predicted heating load. This effect is extremely small and not statistically significant.

Adding `window_age` slightly reduces training MSE from 10.54253 to 10.54208 and slightly increases R-squared from 0.9132390 to 0.9132427. However, adjusted R-squared decreases from 0.9122089 to 0.9120967, and AIC worsens from 4010.450 to 4012.418. More importantly, CV MSE increases from 10.82843 to 10.85146. Therefore, `window_age` does not improve prediction on unseen data, so the simpler model without `window_age` is preferred.